

Desarrollo del Proyecto

Paso a paso

En este archivo se detallará paso a paso la resolución de la preentrega del proyecto e-commerce. Primero se creó el repositorio local, luego el de GitHub para su posterior vinculación. Seguimos con la creación de una estructura base donde destaco el index.html, README.md, y las carpetas ordenadas de forma escalable donde se guardará lo relacionado a css, otras páginas secundarias y archivos multimedia (tanto imágenes como video).

Dentro del index.html, primero codifiqué la estructura general de header, main y footer. En el header incluí la barra de navegación principal (nav). En el main armé la estructura de lo que serían cada una de las películas, separándolas en diferentes etiquetas article independientes y colocando sus respectivas imágenes de póster, nombres, géneros y precio. Implementé un bloque aside al principio de la página de estrenos, configurando un video promocional de fondo en bucle. En el footer, separé en diferentes secciones el contacto, las redes sociales, la información legal y, al final, una etiqueta dedicada para los derechos de copyright.

Luego continué con el diseño de la página en css. Previamente había clasificado cada elemento del archivo HTML con sus respectivas clases para así poder organizar el diseño de cada parte de la página de manera limpia. Implementé un solo archivo css donde fui armando todos los diseños que se me fueron ocurriendo.

El primer diseño fue el de la página principal, del cual se desprenderían el header (barra de navegación) y el footer para las demás páginas, ya que no varían en estructura ni diseño. Las vistas secundarias se organizaron dentro de una carpeta independiente llamada 'pages', enlazándolas todas mediante el uso correcto de rutas relativas (como ../) para asegurar que la navegación no se rompa.

El armado del diseño del inicio y las demás páginas (que alcancé a desarrollar, como la sección de estrenos y la de contacto) fue básicamente un prueba y error con ayuda de la inteligencia artificial para finalmente llegar al diseño final que más encajaba con mi idea del sitio web. Hice uso de propiedades como font-family, background-color y font-size, junto a aspectos más avanzados:

- CSS Flexbox: Utilizado para alinear de forma unidimensional los elementos del menú de navegación y distribuir las columnas informativas del footer.
- CSS Grid: Aplicado en el contenedor del catálogo de películas para generar una cuadrícula dinámica y flexible mediante el uso de la propiedad auto-fit.
- Manejo de Multimedia: Implementé la propiedad object-fit: cover para estandarizar el tamaño visual de los pósteres sin deformar las imágenes. Para el video en bucle, utilicé los atributos autoplay y muted indispensables para asegurar que el navegador permita la reproducción automática sin restricciones.
- Formularios de Consulta: Diseñé un formulario de contacto aprovechando las validaciones como los atributos required y type="email".

Finalmente, agregué algo de código usando media queries (@media) al final del archivo css para adaptar mi página a diferentes dispositivos. Esto garantiza que en pantallas más chicas (como teléfonos móviles y tablets) el menú superior se apile de forma ordenada y las columnas de contenido se reorganicen verticalmente sin desbordar los márgenes.

Posteriormente, incorporé la lógica funcional del proyecto utilizando JavaScript. Creé un archivo `main.js` de alcance global para manejar la interactividad de todas las vistas. Primero, desarrollé la validación del formulario de inicio de sesión, asegurando que los campos obligatorios se completen correctamente antes de guardar la sesión activa del usuario. Luego, implementé un catálogo dinámico estructurado como un arreglo de objetos en memoria. Mediante el uso del método `filter()`, logré que la cuadrícula de películas se renderice y se actualice en tiempo real según la categoría seleccionada por el usuario.

A partir de esta base de datos local, desarrollé un sistema completo para el carrito de compras. Conecté los botones de las distintas páginas para que guarden la selección en el `localStorage`, validando previamente la existencia del ID para evitar agregar elementos duplicados. En la vista del carrito, el código lee esta memoria, inyecta las películas seleccionadas en el HTML, calcula el precio total de forma automática y permite tanto eliminar productos como vaciar la memoria al finalizar la compra, manteniendo sincronizado un contador dinámico en el encabezado de todas las páginas.

A modo de cierre, cabe mencionar que no pude desarrollar los requerimientos avanzados referentes a la integración con una base de datos externa ni consumir una API para la carga del catálogo. Debido a la falta de tiempo generada por el cursado y las responsabilidades en la universidad, decidí priorizar la entrega asegurándome de cumplir sólidamente con todos los requerimientos funcionales mínimos de maquetación, interactividad y persistencia local.